

PROJECT Design Documentation

*The following template provides the headings for your Design Documentation. As you edit each section make sure you remove these commentary 'blockquotes'; the lines that start with a > character and appear in the generated PDF in italics but do so only **after** all team members agree that the requirements for that section and current Sprint have been met. **Do not** delete future Sprint expectations.*

Team Information

- Team name: HalfCourt
- Team members
 - Jonah Witte
 - Kayla Van Bortel
 - Max Klot
 - Ryan Richter

Executive Summary

This is a summary of the project.

Purpose

[Sprint 2 & 4] The purpose of our U-fund application is to allow philanthropists to be able to fund the NYS Paws & Claws foundation, and for administrators of the NYS Paws & Claws Foundation to be able to manage the application.

Glossary and Acronyms

[Sprint 2 & 4] Provide a table of terms and acronyms.

Term	Definition
SPA	Single Page
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
API	Application Programming Interface
DAO	Data Access Object
ID	Identifier
UUID	Universally Unique Identifier
REST	Representational State Transfer
OO	Object-Oriented

Requirements

This section describes the features of the application.

In this section you do not need to be exhaustive and list every story. Focus on top-level features from the Vision document and maybe Epics and critical Stories.

Definition of MVP

[Sprint 2 & 4]

In our Minimum Viable Product in Sprint 2, the user is first routed to a login page. If they enter an unrecognized username, a new account is created for them and they are then sent to the home page. If they enter a username that already exists, they are logged in and sent to the home page. If they enter "admin", they are sent a modified "admin" version of the home page. On the home page, you can see a list of needs in the needs cupboard. If you are a helper, you are able to view needs, add needs to your basket, and checkout. As an admin, you are able to modify needs, post new needs, and delete needs.

MVP Features

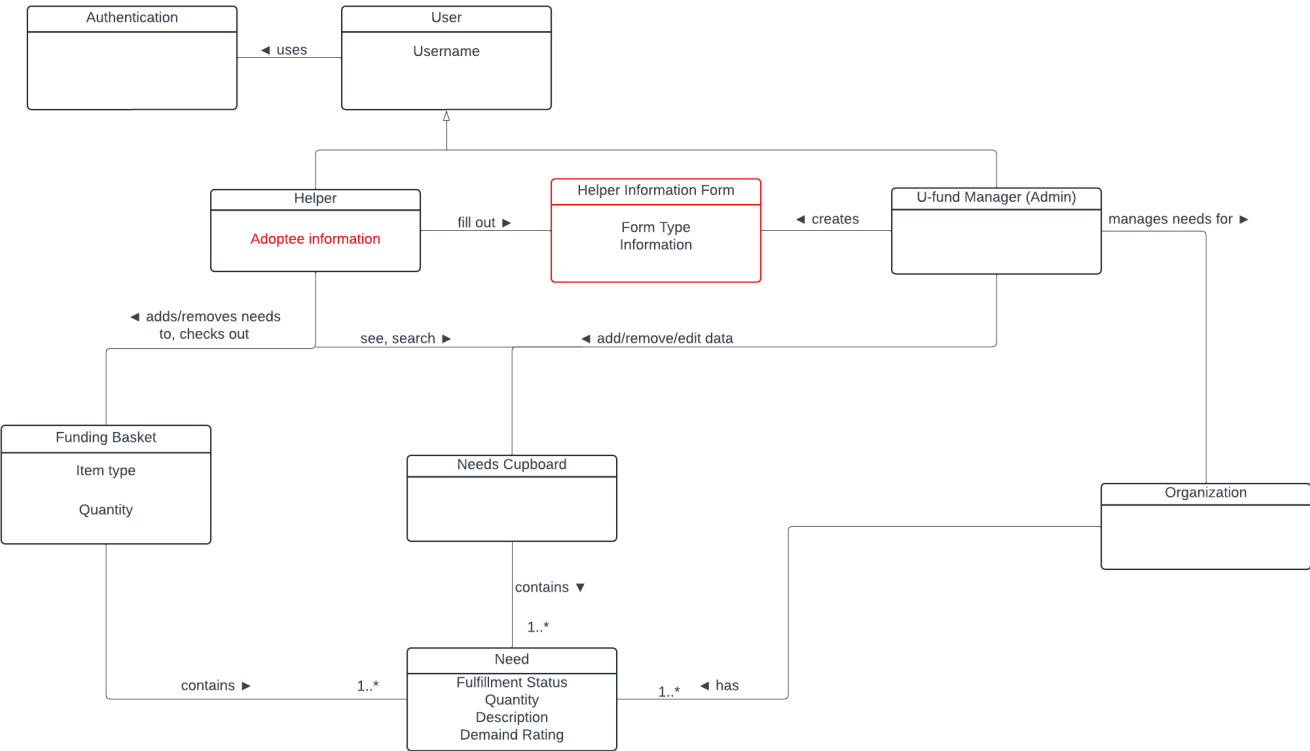
[Sprint 4] Provide a list of top-level Epics and/or Stories of the MVP.

Enhancements

[Sprint 4] Describe what enhancements you have implemented for the project.

Application Domain

This section describes the application domain.



[Sprint 2 & 4] Provide a high-level overview of the domain for this application. You can discuss the more important domain entities and their relationship to each other.

The domain for this application is a New York State based wildlife and animal conservation/refuge organization called NYS Paws and Claws. The application will have:

- Helpers
- Admin
- Needs
- Cupboard
- Baskets

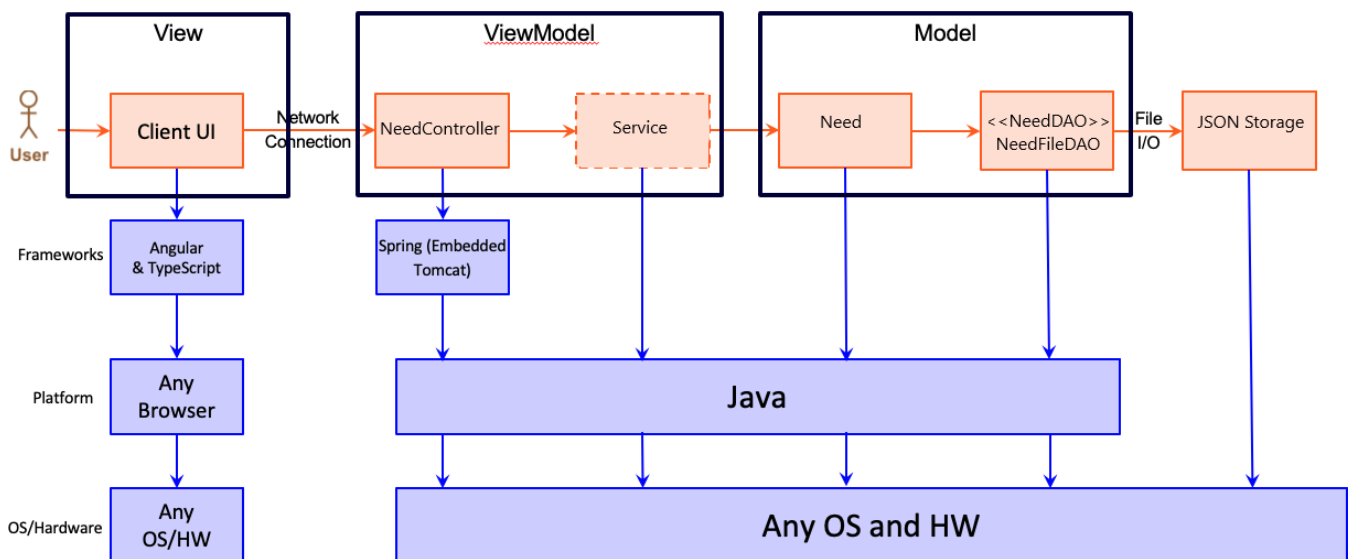
A Helper adds needs to the basket. An Admin edits needs in the cupboard. A Need goes in the cupboard and a basket. A Helper has a basket.

Architecture and Design

This section describes the application architecture.

Summary

The following Tiers/Layers model shows a high-level view of the webapp's architecture. **NOTE:** detailed diagrams are required in later sections of this document.



The web application, is built using the Model–View–ViewModel (MVVM) architecture pattern.

The Model stores the application data objects including any functionality to provide persistence.

The View is the client-side SPA built with Angular utilizing HTML, CSS and TypeScript. The ViewModel provides RESTful APIs to the client (View) as well as any logic required to manipulate the data objects from the Model.

Both the ViewModel and Model are built using Java and Spring Framework. Details of the components within these tiers are supplied below.

Overview of User Interface

This section describes the web interface flow; this is how the user views and interacts with the web application.

On page load, a user is met with a login page, prompting them to enter a username. After entering a username, their username is saved to the application (if not admin), and a new account is created for the user. Then, a user is redirected to the cupboard page, where they can search, select, and add needs to their basket. They can also access their basket with a button, where they can edit quantities of needs within their basket. If a user logs in as admin, they are able to edit the needs cupboard and cannot view the funding basket. If at any time a user refreshes their page, their authentication is lost and they will have to sign in again.

View Tier

[Sprint 4] Provide a summary of the View Tier UI of your architecture. Describe the types of components in the tier and describe their responsibilities. This should be a narrative description, i.e. it has a flow or "story line" that the reader can follow. **[Sprint 4]** You must provide at least **2 sequence diagrams** as is relevant to a particular aspects of the design that you are describing. (**For example**, in a shopping experience application you might create a sequence diagram of a customer searching for an item and adding to their cart.) As these can span multiple tiers, be sure to include an relevant HTTP requests from the client-side to the server-side to help illustrate the end-to-end flow.

[Sprint 4] To adequately show your system, you will need to present the **class diagrams** where relevant in your design. Some additional tips:

- Class diagrams only apply to the **ViewModel** and **Model** Tier
- A single class diagram of the entire system will not be effective. You may start with one, but will be need to break it down into smaller sections to account for requirements of each of the Tier static models below.
- Correct labeling of relationships with proper notation for the relationship type, multiplicities, and navigation information will be important.
- Include other details such as attributes and method signatures that you think are needed to support the level of detail in your discussion.

ViewModel Tier

[Sprint 1]

The main class for our ViewModel implementation is our NeedsController class. This class serves to interact with the NeedDAO interface.

[Sprint 4] Provide a summary of this tier of your architecture. This section will follow the same instructions that are given for the View Tier above.

At appropriate places as part of this narrative provide **one** or more updated and **properly labeled** static models (UML class diagrams) with some details such as associations (connections) between classes, and critical attributes and methods. (**Be sure** to revisit the Static **UML Review Sheet** to ensure your class diagrams are using correct format and syntax.)

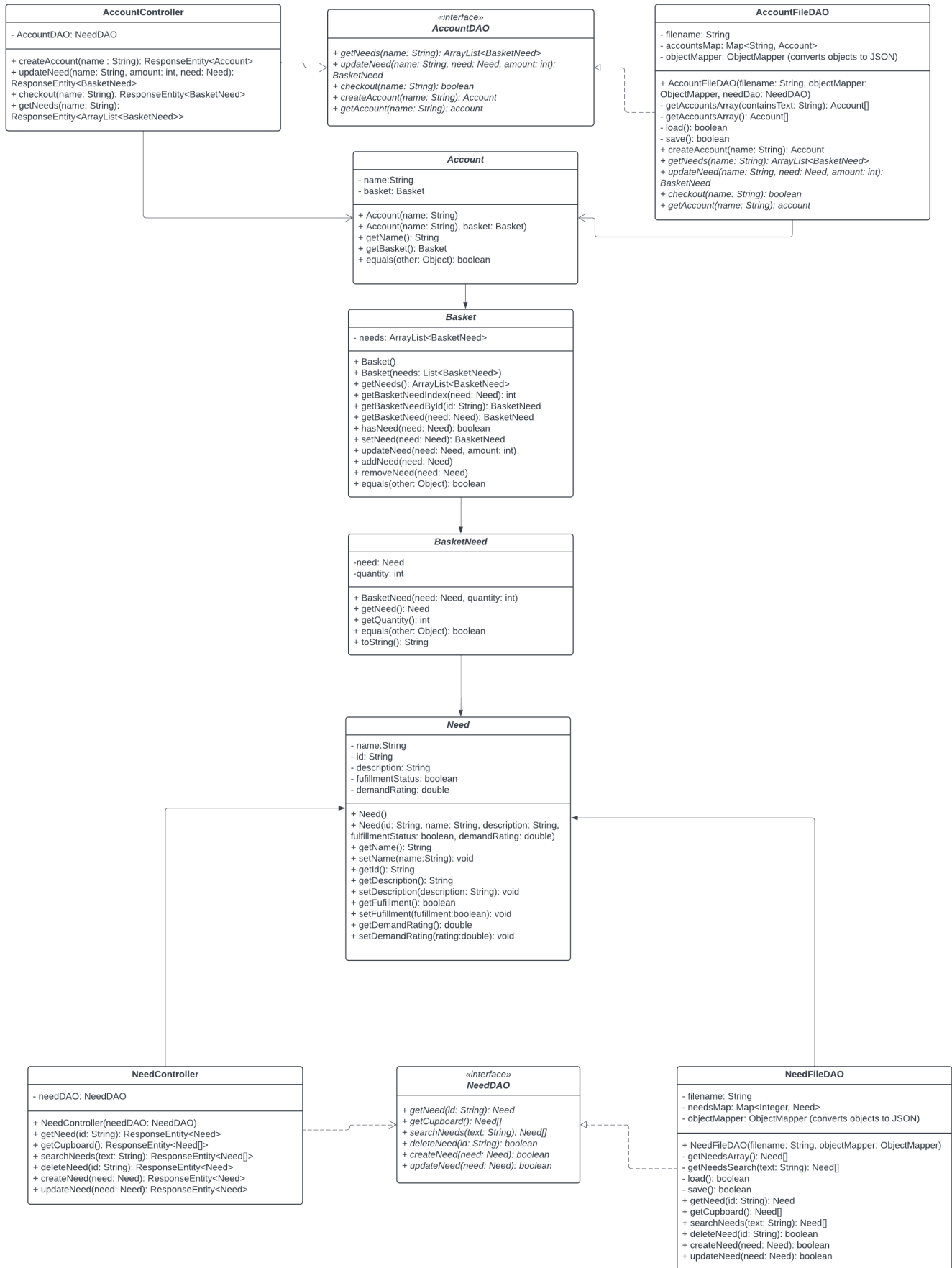
Model Tier

[Sprint 1]

The basic data class for our Model Tier implementation is our Need class. It has generalized members and functions including name, id and fulfillment status.

[Sprint 2, 3 & 4] Provide a summary of this tier of your architecture. This section will follow the same instructions that are given for the View Tier above.

In Sprint 2, we decided to implement three new classes within our model tier, the Account and Basket and BasketNeed classes. The Account class represents an account, which has a name and a basket of needs. The basket of needs contains an arraylist of basketneeds, which are object representations of needs that stay inside a user's basket. Below is our revised UML diagram for sprint 2:



At appropriate places as part of this narrative provide **one** or more updated and **properly labeled** static models (UML class diagrams) with some details such as associations (connections) between classes, and critical attributes and methods. (**Be sure** to revisit the Static **UML Review Sheet** to ensure your class diagrams are using correct format and syntax.)

OO Design Principles

[Sprint 1]

Single Responsibility: Each module should have one tightly focused responsibility.

Open-Close: When modifying a module, you should not make changes to existing logic, but rather consider keeping it and creating new logic instead.

The Single Responsibility

A class is considered to comply with the single responsibility principle if there is one and only one reason for the class to change. One "thing" is not necessarily well defined, but it refers to one group of related behaviors and states.

For example, each of our Java classes has a single function that it conforms to, and we've outlined those responsibilities below to showcase how each class remains independent to its purpose, and doesn't need to change based on other components.

The only class that potentially violates the Single Responsibility is the AccountDAO, which requires changes when either the Account process *or* the basket-checkout process changes, as the checkout functionality requires both an Account basket and a Need cupboard.

NeedController: Only responsible for updating and fetching needs

NeedDAO: Only responsible for DAO processes related to Needs

NeedFileDAO: Same as NeedDAO but specialized

Need: Only needs to be updated if the structure of a need changes

AccountController: Only responsible for creating, updating, and getting Accounts.

AccountDAO: Only responsible for DAO processes related to Accounts (**and** checking out need baskets)

AccountFileDAO: Same as AccountDAO but specialized

Account: Only needs to be updated in the structure of account changes

Basket: Only needs to be updated in the structure of Basket changes

BasketNeed: Only needs to be updated if the structure of BasketNeed changes

Open/Closed

A class should be open to expansion but closed to modification. Expansion means that different components can be used in a class but the behavior of the class does not change. A good example of this is the NeedDAO, which does not need to be modified to add more behavior. Instead, we inherit NeedDAO and create a new object (NeedFileDao) which is an extension.

Our Need class does not comply with this principle. To add new behavior to a Need, we must change all the Needs within the Need database. For example, adding a "type" field to each Need would require us to migrate

the entire database in order to preserve our data while supporting the new value of Need. We would also have to modify our methods and parsers for creating and testing Needs to support the new field.

Open/Closed is useful for maintaining backwards compatibility. By not modifying existing code, you minimize the chance of breaking old code. Instead you can build new independent functionality on top of the existing code. While the old code can run with what is now a limited feature set, the new code, which depends on new features, is also able to run - removing the need to refactor large parts of code (which depends on the thing being changed) when adding features.

[Sprint 2, 3 & 4] Will eventually address upto **4 key OO Principles** in your final design. Follow guidance in augmenting those completed in previous Sprints as indicated to you by instructor. Be sure to include any diagrams (or clearly refer to ones elsewhere in your Tier sections above) to support your claims.

[Sprint 3 & 4] OO Design Principles should span across **all tiers**.

Static Code Analysis/Future Design Improvements

[Sprint 4] With the results from the Static Code Analysis exercise, **Identify 3-4** areas within your code that have been flagged by the Static Code Analysis Tool (SonarQube) and provide your analysis and recommendations.

Include any relevant screenshot(s) with each area.

[Sprint 4] Discuss **future** refactoring and other design improvements your team would explore if the team had additional time.

Testing

This section will provide information about the testing performed and the results of the testing.

Acceptance Testing

[Sprint 2 & 4] Report on the number of user stories that have passed all their acceptance criteria tests, the number that have some acceptance criteria tests failing, and the number of user stories that have not had any testing yet. Highlight the issues found during acceptance testing and if there are any concerns.

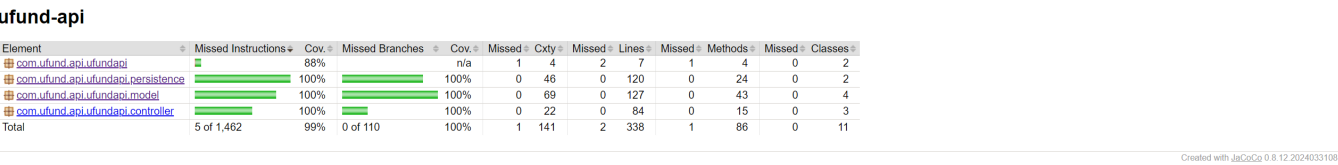
When originally testing the acceptance criteria, we ran into a few problems with specific edge cases (For example, an admin deleting a need and that need staying in a helper's basket). So, we refined our implementation and tests and explored some more edge case tests (negative numbers of Needs, users checking out while the admin was changing Need values, confirmation popups, etc) to ensure the MVP was glitch-free.

Then, after retesting our acceptance criteria, we reached 100% acceptance criteria completion within our acceptance criteria spreadsheet, which is where we now stand as of the current implementation.

Unit Testing and Code Coverage

[Sprint 4] Discuss your unit testing strategy. Report on the code coverage achieved from unit testing of the code base. Discuss the team's coverage targets, why you selected those values, and how well your code coverage met your targets.

[Sprint 2, 3 & 4] Include images of your code coverage report. If there are any anomalies, discuss those.



Anomaly Note: The missing one percent coverage is the main method running the Spring application, which theoretically shouldn't ever fail.

Ongoing Rationale

[Sprint 1, 2, 3 & 4] Throughout the project, provide a time stamp (yyyy/mm/dd): Sprint # and description of any major team decisions or design milestones/changes and corresponding justification.

[Sprint 1] decisions

- (2024/10/1) Switched from integer IDs or unique names for Needs to String UUIDs. This change was agreed upon to simplify the backend logic and limit conflicts based on hidden backend information.
- (2024/10/1) Decided to keep our DAO as Need/NeedFile rather than Cupboard, as the Cupboard name is a concept for the customer and holds no significance to working with the Need classes on the backend.

[Sprint 2] decisions

- (2024/10/4) Need creation now fails with an error response if the numerical arguments (cost, quantity, demand) are less than 0. Given the realistic constraints of needs, those values should never be negative.
- (2024/10/11) Updated UI to use icons instead of text for managing needs buttons
- (2024/10/12) Renamed StorageService to AuthService for user authentication/login functionality
- (2024/10/16) Refactored BasketNeed to have a Need and a quantity instead of a UUID and a quantity
- (2024/10/20) Changed editing need in basket quantity to numerical input instead of increment/decrement methods
- (2024/10/20) Needs are not allowed to be added to the basket if they would overflow the maximum available quantity. We're not sure if the Paws and Claws foundation has the space to store surplus, and we don't want it to go to waste.
- (2024/10/20) All Needs are auto-sorted by Demand rating in the UI
- (2024/10/22) If two users both try to fund the same basket need at the same time and there's not enough to go around, the first one to fund the needs gets to fund them and the later person gets an error message.